1. Selection Sort

Question 1: Trophy Ranking System

A sports event organizer has recorded the scores of 50 participants but needs to display only the top 5 scores on a leaderboard screen. Since only a few top values are needed (not a full sort), use selection sort's "find maximum repeatedly" approach to extract the top 5.

Approach

Instead of sorting the entire array, repeatedly scan the remaining unprocessed portion to find the maximum, move it to the front, and shrink the unprocessed region. Stop after 5 iterations since only the top 5 scores are required. This avoids the cost of sorting all 50 values fully.

Complexity Analysis

Aspect	Complexity	Explanation
Time	O(k × n)	Only k passes are made instead of n, each scanning the remaining n−i elements to find the max.
Space	O(1)	Sorting is done in place; only a copy of the input array is used for safety.

Test Cases

```
assert top_k_scores([72,88,65,90,77,95,60,83,91,68], 5) == [95,91,90,88,83]
assert top_k_scores([5,3,1], 5) == [5,3,1]    # fewer than k items
assert top_k_scores([], 3) == []              # empty input
print('All test cases passed!')
```

Question 2: Memory-Constrained Embedded Device

A low-memory IoT sensor device collects temperature readings and must sort them with minimal write/swap operations, since each write to memory is costly on this hardware. Selection sort performs at most n−1 swaps regardless of input order, making it preferable to bubble or insertion sort here. Implement it with a swap counter to prove the claim.

Approach

Use the standard selection sort algorithm, but only perform a swap when the found minimum is not already in the correct position. Track the number of swaps performed and compare it against n−1 (the theoretical maximum) to demonstrate the low write cost.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Two nested loops compare every pair of elements regardless of input order.
Writes	$\leq n - 1$	A swap only occurs when the minimum found is not already in place, bounding total memory writes.

Test Cases

```
res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])
assert sw <= len(res) - 1                # write bound holds

res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])
assert sw2 == 0                          # already sorted -> zero swaps
print('All test cases passed!')
```

main.py

Share Run

```python
def selection_sort_min_writes(arr):
    arr = arr.copy()
    swaps = 0
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            swaps += 1

    return arr, swaps


# Test Cases
res, sw = selection_sort_min_writes([23.5, 19.2, 25.1, 18.8, 21.4])
assert res == sorted([23.5, 19.2, 25.1, 18.8, 21.4])
assert sw <= len(res) - 1

res2, sw2 = selection_sort_min_writes([1, 2, 3, 4, 5])
assert sw2 == 0

print("All test cases passed!")
```

Output

```
All test cases passed!

=== Code Execution Successful ===
```

Question 3: Library Book Reordering

A librarian wants to rearrange a shelf of 20 books by ID number, but physically moving a book (swap) is time-consuming, while checking labels (comparison) is fast. Model this as an array sorting problem using selection sort and report the number of physical "moves" needed.

Approach

Represent each book as its ID in an array. Apply selection sort, incrementing a move counter only on an actual swap (a physical book relocation), while comparisons (label checks) are unrestricted since they are cheap.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Comparisons dominate the runtime; for n = 20 this is 190 comparisons.
Physical Moves	$\leq n - 1$	Each shelf position is corrected with at most one physical swap.

Test Cases

```
ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
 assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])
 assert moves <= len(ordered) - 1

 ordered2, moves2 = reorder_shelf([100, 200, 300])   # already sorted
 assert moves2 == 0
 print('All test cases passed!')
```

main.py

Share Run Output

```python
def reorder_shelf(arr):
    arr = arr.copy()
    moves = 0
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        if min_index != i:
            arr[i], arr[min_index] = arr[min_index], arr[i]
            moves += 1

    return arr, moves


# Test Cases
ordered, moves = reorder_shelf([305, 102, 250, 118, 199, 400, 101])
assert ordered == sorted([305, 102, 250, 118, 199, 400, 101])
assert moves <= len(ordered) - 1

ordered2, moves2 = reorder_shelf([100, 200, 300])
assert moves2 == 0

print("All test cases passed!")
```

```
All test cases passed!

=== Code Execution Successful ===
```

Question 4: Contest Prize Distribution

In a coding contest, you must repeatedly select the participant with the current highest score, print their name, remove them from the list, and repeat until all participants are ranked. Implement this using selection sort logic without using a separate sorting library.

Approach

Treat this as selection sort in descending order: for each position, scan the remaining participants to find the one with the highest score, swap them into place, and print their name as the next-ranked winner.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Each of the n ranking positions requires a full scan of the remaining participants.
Space	O(1) extra	Ranking is produced in place using swaps; the output list just records the order.

Test Cases

```
ranking = distribute_prizes([('Asha',88), ('Ravi',95), ('Meera',79), ('Dev',95)])
scores_only = [p[1] for p in ranking]
assert scores_only == sorted(scores_only, reverse=True)   # descending order
assert ranking[0][1] == 95                                # top score first
print('All test cases passed!')
```

main.py

Share Run

```python
def distribute_prizes(participants):
    participants = participants.copy()
    n = len(participants)

    for i in range(n - 1):
        max_index = i
        for j in range(i + 1, n):
            if participants[j][1] > participants[max_index][1]:
                max_index = j

        if max_index != i:
            participants[i], participants[max_index] = participants[max_index], participants[i]

    return participants


# Test Cases
ranking = distribute_prizes([('Asha',88), ('Ravi',95), ('Meera',79), ('Dev',95)])

scores_only = [p[1] for p in ranking]
assert scores_only == sorted(scores_only, reverse=True)
assert ranking[0][1] == 95

print("All test cases passed!")
```

Output

```
All test cases passed!

=== Code Execution Successful ===
```

Question 5: Small Dataset in a Mobile App

opping app needs to sort a "recently viewed" list of just 8–10 items by price whenever the · opens the screen. Argue whether selection sort is a reasonable choice for such a small, frequently-refreshed list, and implement it with timing to support your answer.

Approach

very small n, the $O(n^2)$ cost of selection sort is negligible in absolute terms (a handful of oseconds), and its simplicity and low swap count make it a reasonable, easy-to-maintain choice rather than pulling in a general-purpose sort for 8–10 items.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$, $n \leq 10$	For such small n, absolute execution time is in the microsecond range, so complexity class is not a practical concern.
Space	O(1)	In-place sort, no extra memory beyond the working copy.

Test Cases

```
: selection_sort([499,129,899,45,275,60,310,150]) ==
l([499,129,899,45,275,60,310,150])
't selection_sort([]) == []                    # empty list
't selection_sort([7]) == [7]                  # single item
'All test cases passed!')
```

main.py

Share Run

```python
def selection_sort(arr):
    arr = arr.copy()
    n = len(arr)

    for i in range(n - 1):
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr


# Test Cases
assert selection_sort([499,129,899,45,275,60,310,150]) == [45,60,129,150,275,310,499,899]
assert selection_sort([]) == []
assert selection_sort([7]) == [7]

print("All test cases passed!")
```

Output

```
All test cases passed!

=== Code Execution Successful ===
```

2. Bubble Sort

Question 1: Exam Result Slip Correction

A teacher has a nearly sorted list of student roll numbers (only 2–3 are out of place due to late corrections). Use optimized bubble sort (with an early-exit flag) to fix the order and show it takes very few passes.

Approach

Run bubble sort with a 'swapped' flag that tracks whether any swap occurred in a pass. If a pass completes with no swaps, the list is already sorted and the algorithm exits early — ideal for nearly sorted data like this.

Complexity Analysis

Aspect	Complexity	Explanation
Best Case	O(n)	One pass detects a fully sorted array and exits when no swaps occur.
This Scenario	≈ O(n)	Only 2–3 misplaced roll numbers means very few passes are needed before the flag stays false.

Test Cases

```
sorted_rolls, passes = optimized_bubble_sort([101,102,104,103,105,107,106,108])
assert sorted_rolls == sorted([101,102,104,103,105,107,106,108])
assert passes < 8                          # fewer than full n passes

sorted_ok, passes_ok = optimized_bubble_sort([1,2,3,4,5])   # already sorted
assert passes_ok == 1                      # exits after first pass
print('All test cases passed!')
```

main.py

Share Run

```python
def optimized_bubble_sort(arr):
    arr = arr.copy()
    n = len(arr)
    passes = 0

    for i in range(n - 1):
        swapped = False
        passes += 1

        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True

        if not swapped:
            break

    return arr, passes


# Test Cases
sorted_rolls, passes = optimized_bubble_sort([101,102,104,103,105,107,106,108])
assert sorted_rolls == sorted([101,102,104,103,105,107,106,108])
assert passes < 8

sorted_ok, passes_ok = optimized_bubble_sort([1,2,3,4,5])
assert passes_ok == 1

print("All test cases passed!")
```

Output

```
All test cases passed!

=== Code Execution Successful ===
```

Question 2: Traffic Signal Priority Queue

At an intersection, vehicle priorities (ambulance > bus > car) arrive in a small queue and need constant reordering as new vehicles arrive. Simulate this using bubble sort each time a new vehicle is added, and discuss whether bubble sort's simplicity outweighs its inefficiency here.

Approach

Assign a numeric priority to each vehicle type. Whenever a vehicle joins the queue, append it and re-run bubble sort on the (small) queue to restore priority order. Since the queue is small and rarely has more than a handful of vehicles, bubble sort's $O(n^2)$ cost is negligible, and its simplicity makes it easy to reason about and verify in a safety-critical system.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Re-sorting the whole small queue on every arrival is acceptable since n stays very small.
Trade-off	Simplicity vs. efficiency	Bubble sort is easy to verify for correctness in a safety-critical setting, which outweighs its inefficiency at this scale.

Test Cases

```
queue = ['car', 'car', 'bus']
 queue.append('ambulance')
 result = bubble_sort_queue(queue)
 assert result == ['ambulance', 'bus', 'car', 'car']
```

```
assert bubble_sort_queue(['ambulance']) == ['ambulance']   # single vehicle
print('All test cases passed!')
```

main.py

Share Run

```python
def bubble_sort_queue(queue):
    queue = queue.copy()

    priority = {
        "ambulance": 1,
        "bus": 2,
        "car": 3
    }

    n = len(queue)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            if priority[queue[j]] > priority[queue[j + 1]]:
                queue[j], queue[j + 1] = queue[j + 1], queue[j]

    return queue


# Test Cases
queue = ['car', 'car', 'bus']
queue.append('ambulance')
result = bubble_sort_queue(queue)
assert result == ['ambulance', 'bus', 'car', 'car']

assert bubble_sort_queue(['ambulance']) == ['ambulance']

print("All test cases passed!")
```

Output

```
All test cases passed!

=== Code Execution Successful ===
```

Question 3: Bubble Sort Visualization Tool

You are building an educational tool to visually demonstrate how "bubbles" (largest elements) rise to the top with each pass. Implement bubble sort so it prints/returns the state of the array after every single pass, useful for animation.

Approach

Modify standard bubble sort so that after each outer-loop pass, the current state of the array is stored (or printed) as a frame. These frames can then be fed into an animation or step-through UI.

Complexity Analysis

Aspect	Complexity	Explanation
Time	$O(n^2)$	Same as standard bubble sort; storing frames adds O(n) work per pass.
Space	$O(n^2)$ worst case	Up to n frames are stored, each of size n, useful for driving an animation.

Test Cases

```
frames = bubble_sort_with_frames([5, 1, 4, 2, 8])
assert frames[-1] == sorted([5, 1, 4, 2, 8])  # final frame is sorted
assert frames[0] == [5, 1, 4, 2, 8]           # first frame is the original input
assert len(frames) >= 2                       # at least an initial and final frame
print('All test cases passed!')
```

main.py

Share Run

```python
def bubble_sort_with_frames(arr):
    arr = arr.copy()
    frames = [arr.copy()]
    n = len(arr)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
        frames.append(arr.copy())

    return frames


# Test Cases
frames = bubble_sort_with_frames([5, 1, 4, 2, 8])

assert frames[-1] == sorted([5, 1, 4, 2, 8])
assert frames[0] == [5, 1, 4, 2, 8]
assert len(frames) >= 2

print("All test cases passed!")
```

Output

```
All test cases passed!

=== Code Execution Successful ===
```

Question 4: Sorting Sensor Alerts by Severity

A monitoring dashboard receives 15 alerts per minute that must be sorted by severity level (1 = critical to 5 = low) for display. Use bubble sort and measure whether the early-termination optimization meaningfully improves performance on this small, frequently-updated dataset.

Approach

Implement both a plain bubble sort and an optimized version with the early-exit flag, then compare pass counts and comparisons on the same alert data to see how much the optimization helps for this small, often-partially-sorted stream.

Complexity Analysis

Aspect	Complexity	Explanation
Plain Bubble Sort	$O(n^2)$ always	Always runs the full n−1 passes regardless of how sorted the data already is.
Optimized Bubble Sort	O(n) best, $O(n^2)$ worst	Exits early once a pass makes no swaps, which helps when alerts arrive already close to sorted.

Test Cases

```
alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]
r1, c1 = bubble_sort_plain(alerts)
r2, c2 = bubble_sort_optimized(alerts)
assert r1 == r2 == sorted(alerts)     # both produce the same sorted result
assert c2 <= c1                       # optimized never does more comparisons
print('Plain comparisons:', c1, '| Optimized comparisons:', c2)
print('All test cases passed!')
```

main.py

Share Run

```
        for j in range(n - 1 - i):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]

    return arr, comparisons


def bubble_sort_optimized(arr):
    arr = arr.copy()
    n = len(arr)
    comparisons = 0

    for i in range(n - 1):
        swapped = False
        for j in range(n - 1 - i):
            comparisons += 1
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
                swapped = True
        if not swapped:
            break

    return arr, comparisons


# Test Cases
alerts = [2,1,3,2,1,4,3,2,5,1,2,3,4,1,2]

r1, c1 = bubble_sort_plain(alerts)
r2, c2 = bubble_sort_optimized(alerts)

assert r1 == r2 == sorted(alerts)
assert c2 <= c1
```

Output

```
Plain comparisons: 105
Optimized comparisons: 99
All test cases passed!

=== Code Execution Successful ===
```

Question 5: Card Game Hand Sorting

In a card game app, a player's hand of 13 cards must be sorted by rank whenever a new card is drawn. Since only one card is added at a time (nearly sorted data), implement bubble sort and compare its pass count to a full re-sort from scratch.

Approach

Use optimized bubble sort after appending the new card. Because only one card is out of place, the algorithm should need very few passes (proportional to how far the new card must travel) compared to sorting the entire hand from an unsorted state.

Complexity Analysis

Aspect	Complexity	Explanation
Nearly Sorted Hand	≈ O(n)	Only the newly added card needs to "bubble" into place, so few passes are needed.
Fully Shuffled Hand	$O(n^2)$	A randomly shuffled hand requires close to the maximum number of passes and swaps.

Test Cases

```
hand = [2, 4, 6, 8, 9, 11, 13]
hand.append(7)                  # nearly sorted: one new card
final_hand, passes_incremental = bubble_sort_hand(hand)
assert final_hand == sorted(hand)

import random
shuffled = hand.copy()
```

```
random.shuffle(shuffled)
_, passes_full = bubble_sort_hand(shuffled)
assert passes_incremental <= passes_full   # nearly-sorted needs no more passes
print('All test cases passed!')
```